Relatório 25 - Processamento de Linguagem Natural (NLP)

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre NPL. O objetivo da prática foi juntar as duas coisas em um projeto prático sobre papers do arXiv: automatizar a coleta com Airflow, converter os PDFs em texto e treinar um modelo de linguagem que prevê a próxima palavra. O resultado esperado era um modelo treinado, avaliado em papers que ele nunca viu e capaz de gerar texto no estilo científico.

2. Desenvolvimento

O fluxo completo ficou assim:

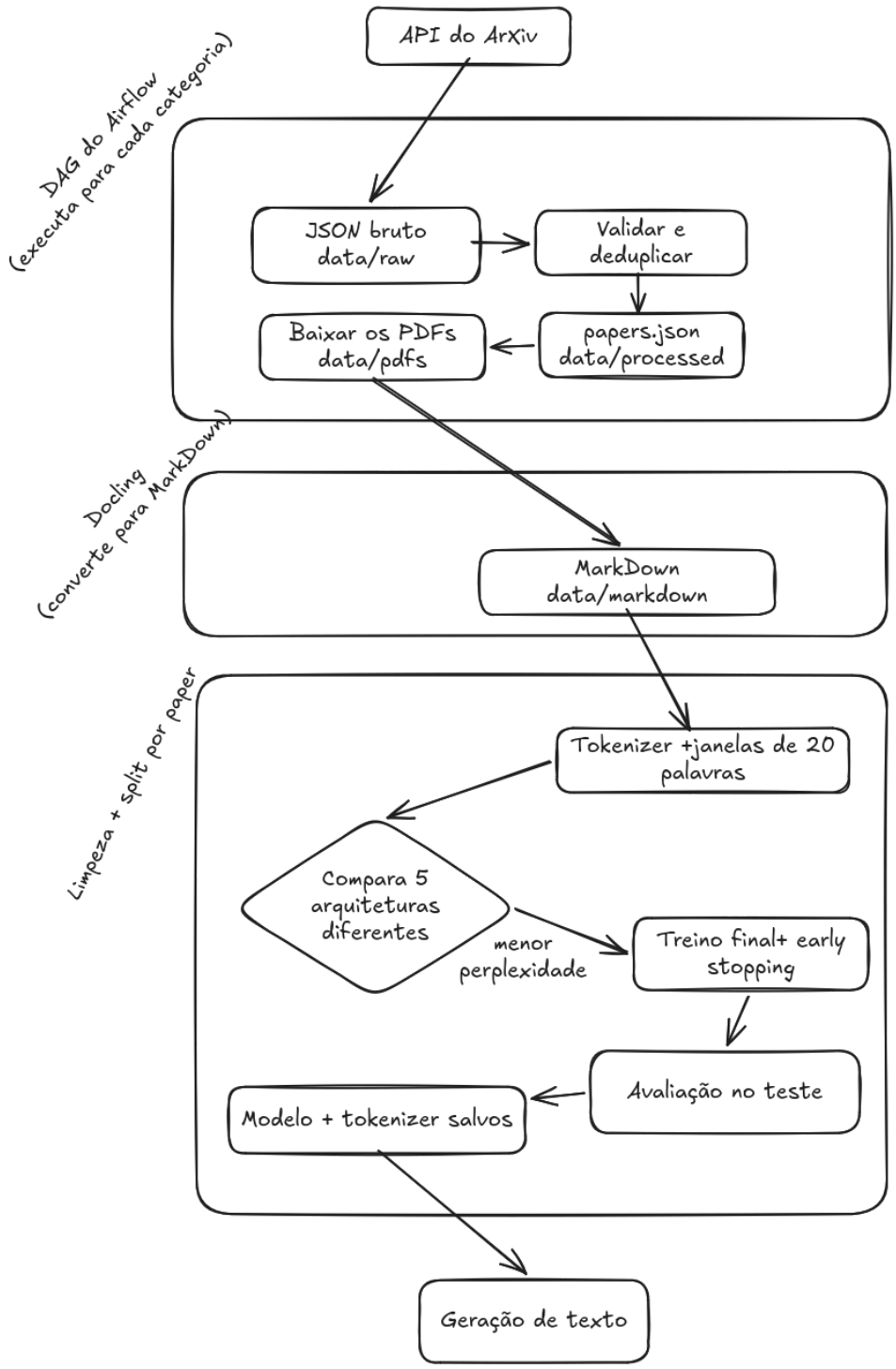


Figura 1. Fluxo completo de execução. (autoria própria)

Extração dos dados (Airflow)

Reutilizei o *pipeline* de dados feito no card Pipeline de Dados 2, alterei algumas coisas:

- **Mais papers na primeira execução:** mudei o start_date para 04/10/2026 e mantive catchup=True. Notei que sem o catchup o start_date não adianta

nada, porque o Airflow só rodaria o último dia. Com max_active_runs=1, os dias atrasados rodam um de cada vez.

- **Download direto do PDF:** criei uma nova *task* baixar_pdfs, que segue o pdf_url de cada paper. Ela confere se o arquivo começa com %PDF (senão o arXiv pode devolver uma página HTML de erro e eu salvaria lixo como .pdf), pula o que já foi baixado e espera 3s entre downloads.
- **Só o caminho vai pelo XCom:** o XCom é para metadados, não para dados. Cada task devolve o caminho do arquivo, nunca a lista de papers.

A primeira execução falhou com HTTP 429 e 503. As três categorias (cs.LG, Cs.AI, cs.CL) rodam em paralelo, e o limite do arXiv (1 requisição a cada 3s) vale para o IP inteiro, não por task. O paralelismo que era vantagem antes estourava o limita da API.

Transformação de PDF para Markdown

Eu tinha um conversor que uso no dia a dia de PDF para Markdown, aproveitei ele para usar na transformação de dados deste card.

- **Docling:** os papers têm duas colunas, e extratores simples misturam a ordem de leitura e geram frases embaralhadas. O Docling detecta as colunas, e essa ordem é necessária para um modelo que aprende “qual palavra vem depois”.
- **OCR desligado:** PDFs do arXiv já tem texto embutido, então o OCR só ia deixar mais lento.
- **Conversor criado uma vez:** carregar os modelos do Docling é muiot caro, então aproveitar o mesmo conversor em todos os PDFs para evitar este custo para cada arquivo.
- Os Markdown já convertidos são pulados, então dá para parar e continuar sem perder trabalho.

Carregamento dos dados

- **Limpeza:** o Docling deixa marcadores de imagem e fórmula, tabelas, símbolos de Markdown e lista de referências, então cortei tudo a apartir de “References”, porque ela é um texto de natureza diferente (nomes, anos, siglas) e o modelo aprenderia a gerar bibliografia em vez de texto científico.
- **Divisão por paper:** se a divisão fosse por janela, trechos vizinhos do mesmo paper cairiam em treino e validação, e a métrica ficaria bonita e falsa. Dividindo por paper, o teste é texto realmente novo.
- **Tokenizer ajustado só no treino:** ajustar no conjunto todo deixaria o teste vazar no vocabulário.
- **num_words=8000 com <OOV>:** com texto de paper o vocabulário passa fácil de dezenas de milhares de palavras, e a saída softmax desse tamanho seria enorme e lenta. FIca só o que é frequente, e o resto vira <OOV>.

- **Janelas deslizantes de 20 palavras:** diferente do n-grama com padding da aula, que gera milhões de zeros em texto longo. Aqui as 20 palavras anteriores são a entrada e a seguinte é o alvo.

Comparação de arquiteturas

Arquitetura	Por que entrou na comparação
embedding_media	Faz a média dos vetores e não tem noção de ordem
lstm	Guarda memória da sequência
bilstm	Lê nos dois sentidos (SCHUSTER; PALIWAL, 1997)
lstm_empilhada	Duas camadas aprendem padrões mais abstratos
gru	Vrsão mais leva da LSTM (CHO et al.,2014)

Todas rodam com o mesmo split, a mesma quantidade de épocas e o mesmo otimizador. Sem isso a comparação não conta, já que qualquer diferença podia vir da configuração e não da arquitetura.

- **Perda sparse_categorical_crossentropy:** é a mesma matemática do categorical_crossentropy da aula, só que o alvo fica como inteiro. O to_categorical com 8000 classes e centenas de milhares de exemplos estouraria a memória.
- **Métrica de escolha, a perplexidade (exp(loss)):** ela diz “entre quantas palavras o modelo está em dúvida”. A acurácia sozinha engana, porque acertar a palavra exata é muito difícil.

Treino

A arquitetura de menor perplexidade na validação é treinada por até 30 épocas com EarlyStopping (paciência 3) e restore_best_weights. Com pouco texto, o modelo começa a decorar rápido: a loss de treino segue caindo enquanto a de validação sobe. O early stopping interrompe nesse ponto e devolve os pesos da melhor época.

Avaliação e geração

O modelo é avaliado no conjunto de teste, formado por papers que não foram usados nem para treinar nem para escolher a arquitetura. também comparei com um baseline trivial (chutar sempre a palavra mais comum).
Resultados com os papers reais:

Arquitetura	Parâmetros	val_loss	Perplexidade	val_acc	val_top5
embedding_media	1844928	6.692286	806.163432	0.073377	0.194827
lstm	2158600	6.195033	490.307661	0.126161	0.253081
bilstm	3509200	6.180918	483.435421	0.126326	0.255189
lstm_empilhada	2339200	6.326477	559.183146	0.113476	0.241647
gru	2121400	6.155904	471.492793	0.128534	0.260791

Modelo escolhido	Loss	Perplexidade	Acurácia	Top-5	Baseline (palavra mais comum)
gru	6.127	458.3	0.122	0.263	0.056

Geração de texto e temperatura: a aula usa argmax, que sempre escolhe a palavra mais provável e tende a cair em repetição. Implementei a temperatura:

Temperatura	Comportamento	Quando usar
0 (argmax)	Determinístico e repetitivo	Var o que o modelo “tem mais certeza”
0.7 a 0.8	Equilibrado	Usa Geral
1.0	Mais variado, mais erros	Explorar variações

temperatura 0 -> in this paper we propose the two benchmarks and the two benchmarks and the highest model training and the original checkpoints in the original checkpoints in appendix c 3 4 4 4 3 3 2

temperatura 0.7 -> in this paper we propose a novel t2i language model to the strongest study for the broader word and the fixed output token per work with the same stage 3 4 we compare the same

temperatura 1.0 -> in this paper we propose transformer evidence and hi jaegle et al 2026 line et al 2024 jia et al 2025 wu et al 2026c claude realize recursive beyond ocr training is the exact

Parte spaCy: NER de domínio nos abstracts

Em paralelo, usei a aula de spaCy em um notebook que reconhece MODEL, DATASET e METRIC nos abstracts. A combinação das técnicas da aula foi o que fez diferença: o EntityRuler entra antes do ner para ter prioridade, uma regex captura versões de modelos (GPT-4, Llama-3.1) e o Matcher extrai relações como “A supera B”. O filter_spans resolve sobreposições. O modelo estatístico puro do spaCy não reconhece essas categorias e rotula nomes de modelos como organizações ou produtos, que é a limitação que as regras corrigem. Esse pipeline não treina nada, eu decidi usar regras, porque a própria aula fala que o treino de NER seria outra parte do curso.

Limitações

O corpus tem poucas centenas de papers, o que é bem pouco para um modelo de linguagem, então o texto gerado tende a ser gramatical mas sem coerência global.
A comparação de arquiteturas usa poucas épocas por modelo, então mostra uma tendência e não um ranking definitivo.
O vocabulário limitado a 8000 palavras transforma termos técnicos raros em <OOV>.

3. Conclusão

A maior parte do trabalho foi antes do modelo: a ordem de leitura do PDF, a limpeza das referências e a divisão por paper pesaram mais na validade dos resultados de que a escolha entre LSTM e GRU. Do lado de engenharia, aprendi que paralelismo não ajuda quando o limite está em uma API externa e que o dono dos volumes do Docker é um detalhe que quebra o pipeline inteiro. Já das aulas, ficou claro que spaCy com regras é rápido, explicável e não precisa de dados rotulados, enquanto o modelo treinado aprende padrões sozinho, mas exige muito mais texto e uma avaliação cuidadosa. Com mais papers e mais épocas, o próximo passo que eu seguiria seria treinar um modelo maior ou fazer fin-tuning de um modelo pré-treinado.

4. Referências

APACHE AIRFLOW. **Apache Airflow Documentation**: DAG runs, catchup and XComs. Disponível em: https://airflow.apache.org/docs/.

ARXIV. **arXiv API User's Manual**. Ithaca: Cornell University, [s.d.]. Disponível em: https://info.arxiv.org/help/api/user-manual.html.

AUER, C. *et al.* **Docling Technical Report**. arXiv:2408.09869, 2024. Disponível em: https://arxiv.org/abs/2408.09869.

CHO, K. *et al.* **Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation**. arXiv:1406.1078, 2014. Disponível em: https://arxiv.org/abs/1406.1078.

HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735-1780, 1997.

MATTINGLY, W. **[Título do vídeo: curso de spaCy]**. YouTube, [ano]. Disponível em: https://www.youtube.com/watch?v=dIUTsFT2MeQ.

MORONEY, L. **Natural Language Processing (NLP) Zero to Hero**. TensorFlow, YouTube, [ano]. Disponível em: https://www.youtube.com/watch?v=fNxaJsNG3-s.

SCHUSTER, M.; PALIWAL, K. K. Bidirectional recurrent neural networks. **IEEE Transactions on Signal Processing**, v. 45, n. 11, p. 2673-2681, 1997.

SPACY. **spaCy Usage Documentation**: EntityRuler, Matcher and custom pipeline components. Disponível em: https://spacy.io/usage.

TENSORFLOW. **Keras API reference**: Tokenizer, Embedding, LSTM, GRU and callbacks. Disponível em: https://www.tensorflow.org/api_docs.